

Álgebra Relacional

Marcos Ricardo Kich

1. Introdução

1.1 Onde estamos

Até este ponto do estudo de bancos de dados, o foco esteve na construção da estrutura. Primeiro aprendemos a observar uma realidade e representá-la por meio de um Modelo Entidade-Relacionamento. Em seguida, transformamos esse modelo em um Modelo Relacional, aplicando regras de mapeamento e normalização para chegar a um conjunto de tabelas consistente e livre de redundâncias desnecessárias.

Um banco de dados bem projetado, porém, só cumpre o seu papel quando permite recuperar as informações certas no momento certo. Armazenar dados é apenas metade do trabalho. A outra metade é conseguir localizá-los, combiná-los e apresentá-los de acordo com cada pergunta que o sistema precisa responder. É nesse ponto que entra a Álgebra Relacional.

1.2 Por que estudar Álgebra Relacional

A Álgebra Relacional é uma linguagem formal para consultar bancos de dados relacionais. Ela descreve, por meio de um pequeno conjunto de operações, como obter um resultado a partir das tabelas existentes. Na prática do dia a dia, quem consulta um banco usa SQL, uma linguagem de mais alto nível executada pelo Sistema Gerenciador de Banco de Dados. Então por que estudar a álgebra antes?

A resposta é que a Álgebra Relacional ensina a pensar a consulta. Ela expõe, de forma clara e sem os detalhes de sintaxe do SQL, o raciocínio que toda consulta segue: quais tabelas participam, como elas se ligam, quais linhas interessam e quais colunas devem aparecer. Quem entende esse raciocínio aprende SQL com muito mais facilidade, porque passa a enxergar o comando como a tradução de uma ideia que já sabe formular. Além disso, é a Álgebra Relacional que está por trás da forma como os próprios SGBDs processam e otimizam as consultas internamente.

1.3 Objetivos deste material

Ao final deste material, espera-se que você seja capaz de:

- reconhecer as operações fundamentais da Álgebra Relacional e o que cada uma faz;
- ler e escrever expressões que combinam essas operações para responder a uma pergunta;
- resolver uma consulta de forma estruturada, decidindo a ordem das operações;
- relacionar cada operação da álgebra ao seu equivalente em SQL.

1.4 A base de dados utilizada

Para manter a atenção nos conceitos, e não em uma estrutura nova, todos os exemplos usam a mesma base de uma companhia aérea trabalhada ao longo do estudo. Ela é formada pelas relações a seguir.

Relação	Atributos	Chave
Modelo	id_modelo, nome, num_lugares, autonomia	id_modelo
Aviao	matricula, id_modelo, data_fabricacao	matricula
Piloto	num_licen, nome	num_licen
Voo	num_voo, data_partida, hora, partida, destino, matricula, num_licen	num_voo
AptoPilotar	id_modelo, num_licen	(id_modelo, num_licen)
Dependente	id_dependente, nome, data_nasc, num_licen	id_dependente

As ligações entre as tabelas seguem as chaves estrangeiras: um avião pertence a um modelo (Aviao.id_modelo); um voo é realizado por um avião e por um piloto (Voo.matricula e Voo.num_licen); um dependente está associado a um piloto (Dependente.num_licen); e AptoPilotar registra quais pilotos estão habilitados a operar quais modelos. Guarde essa estrutura, pois voltaremos a ela em cada operação.

2. O que é a Álgebra Relacional

2.1 Relações, atributos e tuplas

Antes das operações, vale alinhar três termos que aparecem o tempo todo. Uma relação é uma tabela: o conjunto de dados sobre um mesmo assunto, como Piloto ou Voo. Cada linha da tabela é uma tupla, ou seja, um registro individual, como um piloto específico. Cada coluna é um atributo, uma propriedade daquele assunto, como o nome do piloto ou o destino do voo. A Álgebra Relacional opera sobre relações inteiras, não sobre linhas isoladas.

2.2 Uma linguagem formal e fechada

Duas características tornam a Álgebra Relacional simples e poderosa ao mesmo tempo.

A primeira é ser uma linguagem formal. Ela não é executada diretamente por um computador da forma como o SQL é. Seu propósito é descrever consultas com precisão matemática, servindo de base conceitual. Vale um esclarecimento: existem ferramentas de estudo, como o RELAX, que avaliam expressões da Álgebra Relacional para fins de aprendizagem. Isso não contradiz a ideia de linguagem formal. Ferramentas assim funcionam como um laboratório onde você digita a expressão e vê o resultado, o que ajuda a compreender cada operação.

A segunda característica é o fechamento. Toda operação recebe uma ou mais relações como entrada e produz uma nova relação como saída. Como a saída é sempre uma relação, o resultado de uma operação pode servir de entrada para a próxima. É isso que permite construir consultas complexas encadeando operações simples, exatamente como em uma expressão aritmética, na qual o resultado de uma conta alimenta a seguinte.

Relação(ões)	->	Operação	->	Nova relação
--------------	----	----------	----	--------------

2.3 Visão geral das operações

Este material aborda as operações reunidas no quadro abaixo. As três primeiras, seleção, projeção e junção, são de longe as mais frequentes no dia a dia e sustentam a maioria das consultas.

Operação	Símbolo	O que faz
Seleção	σ	Escolhe as linhas que atendem a uma condição.
Projeção	π	Escolhe as colunas desejadas.
União	$\cup$	Reúne as linhas de duas relações compatíveis.
Interseção	$\cap$	Mantém as linhas presentes nas duas relações.
Diferença	$-$	Mantém as linhas da primeira que não estão na segunda.
Produto cartesiano	$\times$	Combina cada linha de uma relação com todas as da outra.
Junção	$\bowtie$	Relaciona linhas de tabelas diferentes por um critério.

2.4 Pensar antes de consultar

Escrever uma consulta fica muito mais fácil quando se responde, antes, a algumas perguntas simples:

- Qual informação eu preciso obter?
- Em quais tabelas essa informação está guardada?
- Preciso combinar dados de mais de uma tabela? Se sim, o que as liga?
- Quais linhas devem permanecer no resultado?
- Quais colunas realmente precisam aparecer?

Responder a isso antes de digitar qualquer símbolo evita a maior parte dos erros e transforma a consulta em uma sequência natural de decisões. Voltaremos a esse roteiro na Seção 5.

3. Operações fundamentais

3.1 Seleção

A **seleção filtra linhas**. Ela percorre uma relação e mantém apenas as tuplas que satisfazem uma condição, sem alterar as colunas. **É um recorte horizontal da tabela**. O símbolo é a letra grega sigma, e a condição é escrita junto ao símbolo.

Para listar os voos que partem de Porto Alegre:

```
σ partida = 'Porto Alegre' (Voo)
```

A entrada é a tabela Voo inteira. A saída é uma nova relação, com os mesmos atributos de Voo, contendo apenas as linhas que atendem à condição:

num_voo	partida	destino	num_licen
V123	Porto Alegre	São Paulo	123456
V001	Porto Alegre	São Paulo	123456

A **condição pode usar os operadores relacionais de comparação (=, <>, <, <=, >, >=)**. Para listar os modelos de aeronave com autonomia maior que 3500 quilômetros:

```
σ autonomia > 3500 (Modelo)
```

A **condição também pode combinar comparações com os operadores lógicos E e OU**. Para encontrar os voos que partem de Salvador com destino a Recife, as duas condições precisam valer ao mesmo tempo:

```
σ partida = 'Salvador' E destino = 'Recife' (Voo)
```

3.2 Projeção

Enquanto a seleção escolhe linhas, **a projeção escolhe colunas**. Ela mantém, de cada tupla, apenas os atributos indicados, produzindo um recorte vertical da tabela. O símbolo é a letra grega pi, seguida da lista de atributos desejados.

Para exibir apenas o nome dos pilotos:

```
|  $\pi$  nome (Piloto)
```

Para listar o número e o destino de cada voo:

```
|  $\pi$  num_voo, destino (Voo)
```

Há um detalhe importante herdado da teoria de conjuntos: a projeção elimina linhas duplicadas do resultado. Se, ao descartar colunas, duas tuplas ficarem idênticas, apenas uma permanece. Por exemplo, π destino (Voo) devolve cada cidade de destino uma única vez, mesmo que vários voos cheguem à mesma cidade.

3.3 Combinando seleção e projeção

Na prática, as duas operações costumam andar juntas: primeiro filtram-se as linhas de interesse, depois mostram-se apenas as colunas necessárias. Para listar somente o número dos voos que partem de Porto Alegre:

```
|  $\pi$  num_voo (  $\sigma$  partida = 'Porto Alegre' (Voo) )
```

Leia de dentro para fora. A seleção interna produz uma relação apenas com os voos de Porto Alegre; a projeção externa recebe essa relação e mantém somente o atributo num_voo. Esse encadeamento é a propriedade de fechamento em ação, e é o padrão da maioria das consultas.

3.4 Operações de conjunto: união, interseção e diferença

União, interseção e diferença vêm diretamente da teoria de conjuntos e tratam relações como conjuntos de tuplas. Para que qualquer uma delas seja aplicada, as duas relações precisam ser compatíveis para união: ter o mesmo número de atributos, com domínios correspondentes na mesma ordem. Em outras palavras, as duas relações precisam ter o mesmo formato.

Nos exemplos a seguir, observe que a base guarda, em Voo, dois atributos que representam cidades: a cidade de partida e a de destino. Podemos projetar cada um deles isoladamente e, por serem do mesmo domínio, aplicar operações de conjunto entre eles.

União. Reúne todas as tuplas das duas relações, eliminando duplicatas. Para obter o conjunto de todas as cidades que aparecem em algum voo, seja como origem, seja como destino:

```
|  $\pi$  partida (Voo)  $\cup$   $\pi$  destino (Voo)
```

Interseção. Mantém apenas as tuplas presentes nas duas relações. Para descobrir as cidades que funcionam tanto como origem quanto como destino:

```
|  $\pi \text{ partida (Voo)} \cap \pi \text{ destino (Voo)}$ 
```

Diferença. Mantém as tuplas da primeira relação que não aparecem na segunda. A ordem importa. Para encontrar as cidades que só aparecem como origem, nunca como destino:

```
|  $\pi \text{ partida (Voo)} - \pi \text{ destino (Voo)}$ 
```

Na base atual, essa última consulta devolve Porto Alegre, pois nenhum voo tem essa cidade como destino. Trocar a ordem dos operandos responderia a outra pergunta: as cidades que só aparecem como destino.

4. Produto cartesiano e junções

As operações desta seção existem para responder a perguntas cujas respostas estão espalhadas em mais de uma tabela. Esse é o caso mais comum em bancos reais, já que a normalização justamente distribui a informação entre tabelas relacionadas.

4.1 Produto cartesiano

O produto cartesiano combina cada tupla de uma relação com todas as tuplas de outra. Se uma relação tem 50 linhas e a outra tem 30, o resultado tem 1500 linhas, com todas as combinações possíveis. O símbolo é o de multiplicação.

O problema é que a maioria dessas combinações não faz sentido. Ao cruzar Piloto com Voo sem nenhum critério, cada piloto acaba associado a todos os voos, inclusive aos que ele não realizou:

```
| Piloto × Voo
```

Sozinho, o produto cartesiano raramente é o que se deseja. Sua importância está em ser o primeiro passo da junção: ele coloca os dados das duas tabelas lado a lado, e cabe a uma condição, na sequência, manter apenas as combinações válidas.

4.2 Junção

A junção acrescenta ao produto cartesiano uma condição que preserva somente as combinações corretas. Na base da companhia aérea, um voo se liga ao seu piloto pelo atributo `num_licen`, que é chave primária em `Piloto` e chave estrangeira em `Voo`. As linhas com significado são aquelas em que esses valores coincidem.

Podemos escrever isso em duas etapas, produto cartesiano seguido de seleção:

```
σ Piloto.num_licen = Voo.num_licen (Piloto × Voo)
```

Essa combinação de produto cartesiano mais seleção pela condição de ligação é tão frequente que ganhou um operador próprio, a junção, representada pelo símbolo de duas peças que se encaixam. A mesma consulta fica:

```
Piloto ⋈ Piloto.num_licen = Voo.num_licen Voo
```

O resultado reúne, em uma só relação, os dados do piloto e os dados do voo correspondente, apenas para as combinações em que o número de licença bate.

4.3 Junção natural

Há um caso particular muito conveniente. Quando as duas relações têm um ou mais atributos de mesmo nome, e esses atributos representam exatamente o vínculo entre elas, a condição pode ser omitida. A junção natural assume que a ligação se dá pelos atributos de mesmo nome e ainda evita repetir a coluna de ligação no resultado.

Como `Piloto` e `Voo` compartilham o atributo `num_licen`, e é por ele que se relacionam, basta escrever:

```
Piloto ⋈ Voo
```

O resultado é equivalente ao da junção anterior, de forma mais enxuta.

Um cuidado importante. A junção natural liga as tabelas por todos os atributos de mesmo nome. Isso é uma armadilha quando existe um nome em comum que não representa o vínculo desejado. As relações `Piloto` e `Dependente` ilustram bem o problema: além de `num_licen`, ambas possuem o atributo `nome`. Uma junção natural entre elas tentaria casar pilotos e dependentes que tivessem, ao mesmo tempo, o mesmo número de licença e o mesmo nome, o que quase nunca ocorre, e o resultado viria praticamente vazio.

Nesses casos, deve-se abandonar a junção natural e declarar a condição de forma explícita, indicando apenas o atributo que de fato liga as tabelas:

```
Piloto ⋈ Piloto.num_licen = Dependente.num_licen Dependente
```

A regra prática é simples: **use a junção natural quando os atributos de mesmo nome forem exatamente o elo entre as tabelas; caso contrário, especifique a condição.**

4.4 Junção externa

A junção comum descarta as linhas que não encontram par. Em muitas situações, porém, queremos manter também os registros sem correspondência. Suponha a pergunta: liste todos os pilotos e, quando houver, o destino de seus voos, incluindo os pilotos que nunca voaram. Uma junção comum entre Piloto e Voo deixaria de fora justamente os pilotos sem voo, pois eles não têm par na tabela Voo.

A junção externa resolve isso preservando as linhas de um dos lados mesmo sem correspondência, preenchendo com valores nulos os atributos que faltam. Ela tem três variações: à esquerda ($\bowtie$, left outer join), que mantém todas as linhas da relação da esquerda; à direita ($\bowtie$, right outer join), que mantém todas as da direita; e completa ($\bowtie$, full outer join), que mantém as de ambos os lados.

Para manter todos os pilotos, tenham eles voado ou não:

```
Piloto ⋈ Voo
```

Os pilotos com voos aparecem normalmente, com os dados do voo preenchidos. Os pilotos sem nenhum voo também aparecem, mas com os atributos vindos de Voo, como destino, marcados como nulos. A partir daí, uma seleção pelo valor nulo isola justamente quem nunca voou:

```
 $\pi$  nome (  $\sigma$  destino é nulo ( Piloto ⋈ Voo ) )
```



Essa é uma forma direta de responder a perguntas do tipo "quais elementos de um lado não têm correspondência do outro", muito comum em relatórios.

5. Construindo consultas

Consultas reais quase sempre combinam várias operações. A boa notícia é que existe um roteiro que funciona para a grande maioria dos casos.

5.1 Uma estratégia passo a passo

1. Identifique qual informação deve ser apresentada ao usuário.
2. Determine em quais tabelas essa informação está guardada.
3. Verifique como essas tabelas se conectam e faça as junções necessárias.
4. Aplique as seleções para manter apenas as linhas desejadas.
5. Por último, use a projeção para deixar somente as colunas que devem aparecer.

Vale destacar o passo cinco. Deixe a projeção para o final. Enquanto constrói a consulta, é útil manter todas as colunas visíveis, pois elas ajudam a conferir se as junções e seleções estão corretas. Só ao final, com o resultado certo, você recorta as colunas que interessam.

5.2 Exemplo resolvido

Pergunta: quais são os nomes dos pilotos que realizaram voos com destino a São Paulo?

Seguindo o roteiro:

- A informação pedida é o nome do piloto, que está na tabela Piloto.
- O destino do voo está na tabela Voo. Logo, precisamos das duas tabelas.
- Elas se ligam por num_licen, então fazemos a junção.
- Filtramos os voos cujo destino seja São Paulo.
- Ao final, projetamos apenas o nome.

A expressão fica:

```
π nome ( σ destino = 'São Paulo' ( Piloto ⋈ Voo ) )
```

Na base atual, os voos com destino a São Paulo são realizados pelos pilotos de licença 123456 e 456789, de modo que o resultado é o conjunto de nomes correspondente:

nome
João Silva
Ana Souza

5.3 Ordem de avaliação

Assim como em uma expressão aritmética, a avaliação segue a precedência dos parênteses: as operações mais internas são executadas primeiro. Na expressão anterior, a ordem é: primeiro a junção `Piloto ⋈ Voo`, depois a seleção pelo destino sobre o resultado dessa junção e, por fim, a projeção do nome. Reorganizar os parênteses muda o que é calculado primeiro e pode alterar o resultado, por isso eles são parte essencial da consulta.

6. Da Álgebra Relacional ao SQL

6.1 Duas linguagens, o mesmo raciocínio

Uma dúvida comum é por que aprender Álgebra Relacional se, no fim, o banco usa SQL. As duas linguagens são complementares: a álgebra descreve o raciocínio da consulta, enquanto o SQL a implementa de forma executável no SGBD. Quem domina a primeira escreve a segunda com naturalidade, porque já sabe o que precisa acontecer.

Álgebra Relacional	SQL
Descreve o que deve ser obtido	Executa a consulta no SGBD
Linguagem formal	Linguagem executável
Base conceitual	Aplicação prática

6.2 Exemplos equivalentes

Os exemplos a seguir mostram a mesma consulta nas duas linguagens, sobre a base da companhia aérea.

Seleção. Quais voos têm destino a São Paulo?

```
 $\sigma$  destino = 'São Paulo' (Voo)
SELECT *
FROM Voo
WHERE destino = 'São Paulo';
```

A cláusula WHERE desempenha o papel da seleção.

Projeção. Quais são os nomes dos pilotos cadastrados?

```
 $\pi$  nome (Piloto)
SELECT nome
FROM Piloto;
```

Os atributos listados após o SELECT correspondem à projeção.

Junção. Quais pilotos realizaram voos com destino a São Paulo?

```
 $\pi$  nome (  $\sigma$  destino = 'São Paulo' ( Piloto  $\bowtie$  Voo ) )
SELECT P.nome
FROM Piloto P
JOIN Voo V ON P.num_licen = V.num_licen
WHERE V.destino = 'São Paulo';
```

A sintaxe muda, mas o caminho é o mesmo: relacionar as tabelas, filtrar as linhas e apresentar apenas as colunas desejadas.

7. Síntese e preparação para SQL

7.1 Quadro-resumo dos operadores

Operação	Símbolo	Função	Equivalente aproximado em SQL
Seleção	σ	Filtra as linhas que atendem a uma condição	WHERE
Projeção	π	Seleciona colunas	SELECT

União	$\cup$	Reúne relações compatíveis	UNION
Interseção	$\cap$	Linhas presentes nas duas relações	INTERSECT
Diferença	$-$	Linhas da primeira que não estão na segunda	EXCEPT ou MINUS
Produto cartesiano	$\times$	Todas as combinações entre duas relações	CROSS JOIN
Junção	$\bowtie$	Relaciona tabelas por um critério	JOIN
Junção externa	$\bowtie \bowtie \bowtie$	Junção que preserva linhas sem correspondência	LEFT, RIGHT, FULL OUTER JOIN

A disponibilidade de INTERSECT, EXCEPT e MINUS varia conforme o Sistema Gerenciador de Banco de Dados utilizado.

7.2 Boas práticas

Ao estudar, procure interpretar cada consulta passo a passo, identificando o papel de cada operação, em vez de decorar expressões prontas. Comece sempre pelo problema: entenda o que se pede, descubra onde os dados estão, ligue as tabelas, filtre e só então recorte as colunas. Esse hábito, além de tornar a Álgebra Relacional mais clara, prepara diretamente o terreno para o SQL.

7.3 Para onde vamos

A Álgebra Relacional oferece os fundamentos conceituais para consultar bancos de dados relacionais. Embora, na prática, os SGBDs executem comandos SQL, o raciocínio construído aqui continuará valendo em todas as consultas que você escrever. Nos próximos estudos, veremos como transformar essas mesmas ideias em comandos SQL executados sobre um banco real, e como uma ferramenta de estudo permite experimentar as operações da álgebra diretamente sobre a base da companhia aérea.